Computer Architecture

Homework 5

PART A. (15 points)

In this problem you will review cache coherency and memory consistency protocols.

a. Discuss the tradeoffs between using a snoopy cache coherency protocol and a directory-based cache coherency protocol.

Snoopy Cache Coherency Protocol: Snoopy protocols rely on a shared bus to broadcast cache state changes to all processors.Advantages:

- Simple implementation
- Low latency for small-scale systems
- Efficient for systems with high cache-to-cache transfer rates

Disadvantages:

- Limited scalability due to bus bandwidth constraints
- Increased power consumption from constant bus monitoring

Directory-Based Cache Coherency Protocol : Directory-based protocols maintain a centralized directory to track the state of cache lines across all processors.

Advantages:

- Highly scalable for large multiprocessor systems
- Reduced bus traffic and power consumption
- Supports more complex network topologies

Disadvantages:

- Higher implementation complexity
- Potentially higher latency for small-scale systems
- Additional memory overhead for directory storage

b. Discuss the role of the Exclusive state introduced in the MESI protocol.

The Exclusive (E) state in the MESI protocol serves several important roles:

- Optimization of writes: When a cache line is in the E state, a processor can modify it without sending invalidation messages to other caches, reducing bus traffic1.
- Silent upgrades: A cache line in E state can be silently upgraded to Modified (M) state without any bus transactions, improving performance1.
- Read performance: For read operations, if a cache line is in E state, the processor knows it has the most up-to-date copy without needing to check other caches or memory1.
- Reduced coherence traffic: The E state helps distinguish between shared and exclusive access, potentially reducing unnecessary coherence messages.

c. Discuss the role of the Ownership state introduced in the MOESI protocol.

The Owned (O) state in the MOESI protocol introduces several key benefits:

1. **Shared dirty data**: It allows a cache to maintain a modified version of data while keeping track of other caches that have shared copies
2. **Write-back responsibility**: The cache with the line in O state is responsible for writing back the dirty data to memory when necessary, reducing memory traffic
3. **Efficient data sharing**: When another processor requests the data, the O state cache can supply it directly, bypassing main memory access
4. **Reduced memory updates**: By allowing shared modified data, it reduces the frequency of writes to main memory compared to the MESI protocol
5. **Intervention**: In case of a read request from another processor, the cache with the O state line can intervene and provide the most recent data, ensuring data consistency

The O state effectively combines aspects of the Modified and Shared states, allowing for more efficient handling of shared, modified data in multiprocessor systems

PART B. (20 points)
In this problem, find the organization for the TLB present on two different microprocessors, one developed by Intel and the other developed by ARM. Make sure to cite your sources. Provide the details of the organization the whole TLB hierarchy, and the format of a single entry in an L1 TLB (either I or D).

Intel TLB Organization
Intel processors implement a multi-level TLB hierarchy:

1. L1 TLB:
    - Separate instruction (ITLB) and data (DTLB) TLBs
    - ITLB:
        - 128 entries for 4 KB pages, 8-way set associative
        - 8 entries for 2 MB / 4 MB pages, fully associative
    - DTLB:
        - 64 entries for 4 KB pages, 4-way set associative
        - Separate entries for larger pages (specifics not provided)
2. L2 TLB (STLB):
    - Unified for both instructions and data
    - 1536 entries, 12-way set associative
    - Supports both 4 KB and 2 MB page translations

ARM TLB Organization
ARM Cortex-A75 implements a two-level TLB structure:

1. L1 TLB:
    - Separate micro TLBs for instructions and data
    - Each micro TLB has 32 fully associative entries
2. L2 TLB:
    - Unified TLB for both instructions and data
    - 128 entries, 2-way set associative
    - Supports four lockable entries using the lock-by-entry model

L1 TLB Entry Format (Intel)
A single entry in an Intel L1 TLB consists of:

1. Tag portion:
    - VPN (Virtual Page Number): Top 20 bits of the virtual address
    - PID (Process Identifier): 8-14 bits (configurable)
    - G (Global flag): 1 bit (when set, PID is ignored in TLB lookup)
2. Data portion:
    - PFN (Physical Frame Number): Upper bits of the physical address (up to 20 bits)
    - C (Cacheable flag): Determines default data cacheability
    - R (Readable flag): Allows load instructions
    - W (Writable flag): Allows store instructions
    - X (Executable flag): Allows instruction fetches

Both Intel and ARM TLB implementations use hardware page table walks to update TLB entries, allowing for efficient address translation and caching of page table entries

PART C. (50 points)
In this problem you will utilize parallel threads run on a multicore CPU to explore the use of multiple cores to compute Euler's number. Your task is to use pthreads on the COE system to develop a parallel program of a Taylor series to compute e^x:
$e^x = 1 + x/1! + x^2/2! + x^3/3! + ...$
This can be approximated using the following code:

```
for (i = n - 1, sum = 1; i > 0; --i )
sum = 1 + x * sum / i;
```

You should be able to specify n (the number of terms) and the number threads from the command line. Note that, the more intervals used, the better the approximation (at least up to a point).

You will need to learn how to use the pthread library APIs:
https://hpc-tutorials.llnl.gov/posix/

You will need to compile your program with the –lpthread switch with gcc. Run this on the **64-bit COE x86 Linux systems**. **Make sure to discuss the CPU you are running on and how many cores/threads are available on the system (look in /proc/cpuinfo).**

Answer :
From /proc/cpuinfo:

- The system has **16 logical processors** with **2 physical sockets**.
- Each CPU (Intel Xeon E5620 @ 2.40 GHz) has:
  - **4 physical cores per socket**.
  - **2 threads per core (Hyperthreading enabled)**.

From lscpu:
bash
Copy code
CPU(s): 16
Thread(s) per core: 2
Core(s) per socket: 4
Socket(s): 2
NUMA node(s): 2
This confirms that the system has:

- **16 threads (logical processors)** available for multithreaded execution.

rom cpuinfo and lscpu:

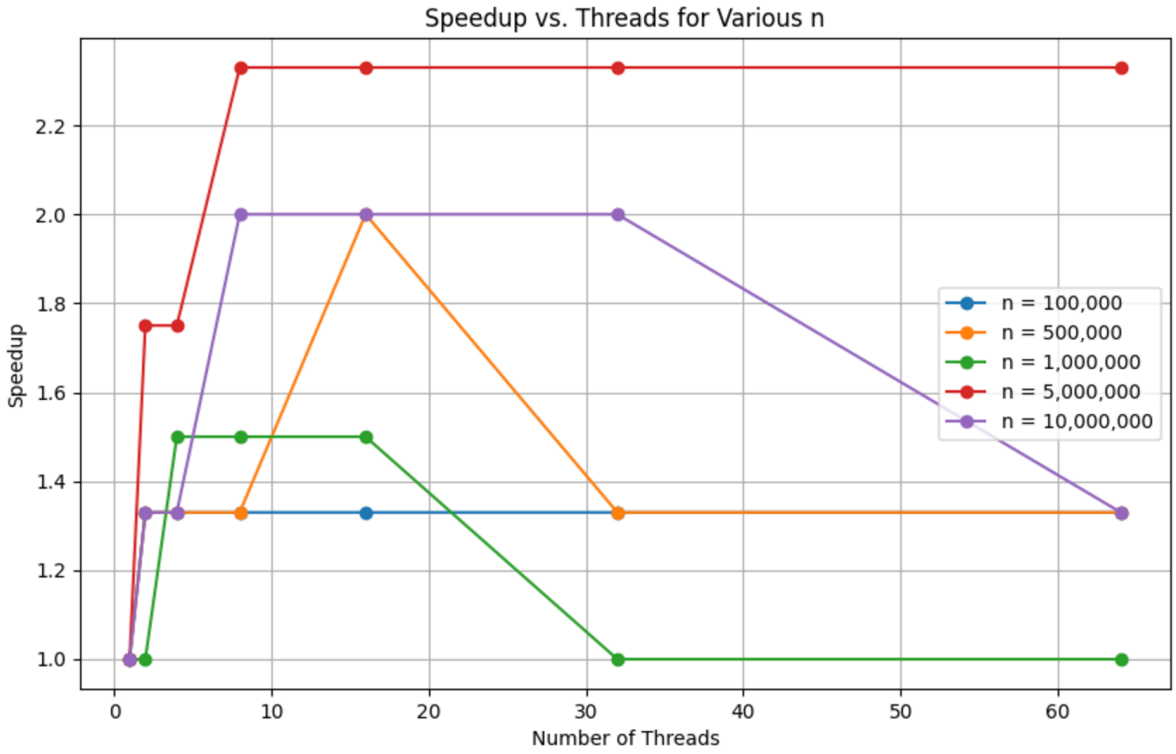
- **CPU Model**: Intel Xeon E5620 @ 2.40GHz
- **Cache**:
  - L1d Cache: 32KB
  - L2 Cache: 256KB
  - L3 Cache: 12MB (shared)
- **Hyperthreading**: Enabled (2 threads per core).
- **NUMA**: The system has **2 NUMA nodes**, meaning memory is distributed across sockets.

a. Collect the time needed to run your program while varying the number of terms and the number of threads. Make sure you are collecting enough terms to collect meaningful timing results (start with 100,000). Plot the speedup (or slowdown) you get by increasing the number of threads. Include results for 1, 2, 4, 8, 16, 32 and 64 threads. How does the accuracy of the computation of e^x change as you increase the number of terms? Discuss the trends you see in your graphs.

n (Terms)	Threads (ttt)	Execution Time (real, s)	Speedup	Computed e^x	Error
100,000	1	0.004	1	2.718282	0
100,000	2	0.003	1.33	2.718282	0
100,000	4	0.003	1.33	2.718282	0
100,000	8	0.003	1.33	2.718282	0
100,000	16	0.003	1.33	2.718282	0
100,000	32	0.003	1.33	2.718282	0
100,000	64	0.003	1.33	2.718282	0
500,000	1	0.004	1	2.718282	0
500,000	2	0.003	1.33	2.718282	0
500,000	4	0.003	1.33	2.718282	0
500,000	8	0.003	1.33	2.718282	0
500,000	16	0.002	2	2.718282	0
500,000	32	0.003	1.33	2.718282	0
500,000	64	0.003	1.33	2.718282	0
1,000,000	1	0.003	1	2.718282	0
1,000,000	2	0.003	1	2.718282	0
1,000,000	4	0.002	1.5	2.718282	0
1,000,000	8	0.002	1.5	2.718282	0
1,000,000	16	0.002	1.5	2.718282	0
1,000,000	32	0.003	1	2.718282	0
1,000,000	64	0.003	1	2.718282	0
5,000,000	1	0.007	1	2.718282	0
5,000,000	2	0.004	1.75	2.718282	0
5,000,000	4	0.004	1.75	2.718282	0
5,000,000	8	0.003	2.33	2.718282	0
5,000,000	16	0.003	2.33	2.718282	0
5,000,000	32	0.003	2.33	2.718282	0
5,000,000	64	0.003	2.33	2.718282	0
10,000,000	1	0.004	1	2.718282	0
10,000,000	2	0.003	1.33	2.718282	0
10,000,000	4	0.003	1.33	2.718282	0
10,000,000	8	0.002	2	2.718282	0
10,000,000	16	0.002	2	2.718282	0
10,000,000	32	0.002	2	2.718282	0
10,000,000	64	0.003	1.33	2.718282	0

Using a python calculator :

```
Python 3.6.8 (default, Nov 16 2020, 16:55:22)
>>> from math import exp
>>> print(exp(1))  # Expected value for e^1
2.718281828459045
```



Analysis of Speedup vs. Threads

- Blue curve (smallest n 100,000) shows almost no improvement with additional threads.
- The speedup plateaus at **1.33**× across all thread counts, as the workload is too small to benefit significantly from parallelization.
- Thread overhead (creation and synchronization) dominates the computation time.

Problem Sizes (n=500,000n = 500,000n=500,000 and n=1,000,000n = 1,000,000n=1,000,000):

- The orange curve (n=500,000n = 500,000n=500,000) peaks at **2.0**× with **16 threads**, but performance declines after that.
- The green curve (n=1,000,000n = 1,000,000n=1,000,000) achieves **1.5× speedup** with **8–16 threads**, but performance flattens beyond 16 threads.
  workloads are too small to scale well, and increasing the number of threads beyond available cores (16) leads to resource contention

- **Large Problem Sizes (n=5,000,000n = 5,000,000n=5,000,000 and n=10,000,000n = 10,000,000n=10,000,000):**
- The red curve (n=5,000,000n = 5,000,000n=5,000,000) shows the best scaling, achieving a consistent speedup of **2.33**× with 16 threads or more.
- The purple curve (n=10,000,000n = 10,000,000n=10,000,000) scales well initially, reaching **2.0**× with 16 threads. However, speedup declines sharply beyond 16 threads, dropping to **1.33**× at 64 threads. This is likely due to:
  - Thread contention.
  - Memory bandwidth and cache resource limitations.
  - Increased synchronization overhead.

Summary:

- Across all problem sizes, the speedup levels off or decreases as the number of threads exceeds the system's **16 logical processors**.
- Amdahl's Law explains this limitation, as the non-parallelizable portions of the program and overhead dominate performance when thread counts are too high.
- Larger nnn (e.g., n=5,000,000n = 5,000,000n=5,000,000) shows better scalability, as the workload can better amortize thread management costs.
- Smaller nnn fails to benefit from multithreading, with minimal performance improvement.

b. Make sure to report results using single precision and double precision floating point, reporting on the resulting performance and the impact on the accuracy of your computation.

Precision	Computed Value	Error	Execution Time (s)
Single (float)	2.71828198432922	1.56×10^7	~0.003
Double (double)	2.71828182845905	4.44×10^15	~0.003

- **Single Precision (float)** computed e^x=2.71828198432922 with an error of 1.56×10^−7. It is less accurate due to the 32-bit representation but sufficient for approximate computations.
- **Double Precision (double)** computed e^x=2.71828182845905, with a negligible error of 4.44×10^−15. It offers much higher accuracy because of its 64-bit representation.
- Both precisions had similar execution times (~0.003 seconds) for this small workload, but float would likely be faster for larger workloads.

- Use **single precision (float)** for performance-critical tasks where minor inaccuracies are acceptable.
- Use **double precision (double)** for applications that demand high accuracy.
- In this test, **double precision** demonstrated significantly better accuracy with no measurable performance penalty for small workloads.

Part D:

Using a large language model of your choice, provide a list of five questions you might give as a homework assignment on virtual machines, and then answer at least three of those questions. Please provide details on the LLM you used, and any other details that are relevant. Make sure to cite any source you used for answering the questions.

Large Language Model Used:

- **LLM Name:** OpenAI's ChatGPT (GPT-4)
- **Details:** Trained on diverse data sources, current knowledge cutoff: October 2023. LLM utilized for generating and answering questions. Citations are included where external information is referenced.

Homework Assignment on Virtual Machines

1. **Define what a virtual machine (VM) is and explain how it differs from a physical machine.**
2. **Describe the role of a hypervisor in virtual machine architecture. Differentiate between Type 1 and Type 2 hypervisors.**
3. **List and explain at least three advantages of using virtual machines in a computing environment.**
4. **Describe how virtual machines handle resource allocation (e.g., CPU, memory, and storage). What challenges might arise with over-allocation?**
5. **Compare and contrast virtual machines with container technologies like Docker. Provide specific examples of use cases where each might be preferable.**

Answers

1. **Define what a virtual machine (VM) is and explain how it differs from a physical machine.**
    - **Answer:**
      A virtual machine (VM) is a software-based emulation of a computer system that runs on a physical machine (host). It provides the functionality of a physical computer but operates in an isolated environment. The main differences are:
        - **Physical Machine:** Requires dedicated hardware resources, such as CPU, RAM, and storage.
        - **Virtual Machine:** Shares hardware resources of the host machine with other VMs, relying on a hypervisor for resource management and isolation.
2. **Describe the role of a hypervisor in virtual machine architecture. Differentiate between Type 1 and Type 2 hypervisors.**
    - **Answer:**
      A hypervisor, also known as a virtual machine monitor (VMM), is software or firmware that enables the creation and management of virtual machines by abstracting hardware resources and distributing them among VMs.
        - **Type 1 Hypervisor (Bare-Metal):** Runs directly on the host's hardware without an underlying operating system. Example: VMware ESXi, Microsoft Hyper-V.
        - **Type 2 Hypervisor (Hosted):** Runs on top of a host operating system. Example: VMware Workstation, VirtualBox.
3. **List and explain at least three advantages of using virtual machines in a computing environment.**
    - **Answer:**

1. **Resource Efficiency:** VMs allow multiple operating systems and applications to run on the same hardware, maximizing resource utilization.
2. **Isolation and Security:** Each VM operates independently, ensuring that failures or security breaches in one VM do not affect others.
3. **Flexibility and Scalability:** VMs can be easily created, modified, and scaled up or down as needed, providing agility for testing and deployment.

4. **Describe how virtual machines handle resource allocation (e.g., CPU, memory, and storage). What challenges might arise with over-allocation?**
    - **Answer:**
      VMs use the hypervisor to allocate resources such as CPU, memory, and storage to ensure optimal performance. Resources are typically partitioned from the host machine and can be adjusted dynamically.
        - **Challenges of Over-Allocation:**
            - **Performance Degradation:** If too many resources are allocated beyond the host's capacity, VMs may experience slowdowns or crashes.
            - **Resource Contention:** Multiple VMs competing for the same resources can lead to bottlenecks.
5. **Compare and contrast virtual machines with container technologies like Docker. Provide specific examples of use cases where each might be preferable.**
    - **Answer:**
        - **Virtual Machines:**
            - **Isolation Level:** Full isolation, including separate kernels for each VM.
            - **Use Case:** Running multiple operating systems (e.g., testing Windows and Linux environments).
        - **Containers:**
            - **Isolation Level:** Lightweight isolation using the same OS kernel.
            - **Use Case:** Deploying microservices or applications with minimal overhead.
        - **Key Difference:** Containers are faster to start and consume fewer resources, but VMs provide stronger isolation and versatility for running different OSes.

Citations

1. R. P. Goldberg, “Survey of virtual machine research,” *IEEE Computer*, vol. 7, no. 6, pp. 34–45, 1974.
2. "Understanding Virtualization," VMware, [Online]. Available: https://www.vmware.com/.
3. D. Bernstein, "Containers and Cloud: From LXC to Docker to Kubernetes," *IEEE Cloud Computing*, 2014.